

ClipMind AI — Project File & Architecture Explanation Script

Comprehensive Codebase & Directory Structure Guide

Project Title: ClipMind AI: Video Summarization & Key Moments Detection Platform **Author:** Adabala Venkata Thrinadh (B.Tech CSE, GMRIT) **Purpose:** Codebase Walkthrough for Technical Evaluators & Reviewers

1. High-Level Folder Structure Overview

```
CLIPMIND AI/ ■■■■ BACKEND/ # Python 3.12 FastAPI Asynchronous REST & WebSocket Server ■■■■ app/ # Application source code (routers, services, models) ■■■■ uploads/ # Working media storage (/videos, /audio, /thumbnails, /exports) ■■■■ Dockerfile # Container definition for backend service ■■■■ requirements.txt # Python package dependencies ■■■■ main.py # ASGI application entry point ■■■■ FRONTEND/ # React 19 + TypeScript + Vite Single-Page Web Application ■■■■ src/ # Frontend source code (components, services, types) ■■■■ package.json # Node.js dependencies & build scripts ■■■■ vite.config.ts # Vite bundler configuration ■■■■ docs/ # Comprehensive Reports, Presentation Deck, and Documentation ■■■■ images/ # Curated system architecture diagrams and screen assets ■■■■ scripts/ # Presentation, codebase, and live demo narration scripts ■■■■ convert_doc.py # Automated markdown-to-DOCX and PDF compiler ■■■■ generate_light_ppt.py # Automated PowerPoint presentation deck generator ■■■■ Dockerfile # Multi-stage production container image definition ■■■■ docker-compose.yml # Multi-service stack orchestration (App + DB) ■■■■ README.md # Quick-start installation and operational guide
```

2. Backend Architecture (BACKEND/)

The backend is built with **FastAPI**, chosen for its native async/await concurrency, automatic OpenAPI documentation, and high-performance ASGI throughput with Uvicorn.

2.1 Core Configuration & Entry Points

- **BACKEND/main.py:**
 - The central ASGI application entry point.
 - Initializes FastAPI with CORS middleware, mounts static media directories (**/uploads**), connects to databases on startup, and registers API routers.
- **BACKEND/app/config.py:**
 - Defines the **Settings** class using Pydantic Settings.
 - Manages environment variables: database connection strings, JWT secret keys, token expiration windows, Google Drive credentials, and upload directory paths.
- **BACKEND/app/database.py:**
 - Configures the dual-persistence database connections:
 1. **SQLAlchemy / Aiosqlite:** Initializes SQLite/PostgreSQL engine for relational user management.
 2. **Motor & Beanie ODM:** Connects to MongoDB Atlas asynchronously and initializes document models (**Video**, **Transcript**, **Summary**, **KeyMoment**, **Quiz**, **FlashcardSet**, **Setting**).
- **BACKEND/app/security.py:**
 - Implements password cryptography via **passlib** with the Bcrypt algorithm (2^{12} work factor).
 - Encodes and decodes signed JSON Web Tokens (JWT) via **python-jose**.
 - Defines the declarative **require_roles(allowed_roles)** dependency function that inspects JWT claims and enforces HTTP 403 blocks for unauthorized access.

2.2 Data Models Layer

- **BACKEND/app/models.py (Relational SQL):**
 - Defines **User** table: primary key ID, email (indexed/unique), full_name, hashed_password, role (**creator**, **learner**, **educator**, **admin**), active status, and timestamp.

- Defines **AuditLog** table: records administrative and system events (actor_id, action_type, details, timestamp).
- **BACKEND/app/mongodb_models.py (Document NoSQL)**:
- Defines Beanie ODM document classes that map to MongoDB collections:
- **Video**: Metadata, storage path, **storage_type**, **drive_file_id**, **drive_web_link**, duration, upload status.
- **Transcript**: Full text and segment arrays with millisecond timestamps, speaker tags, and word tokens.
- **Summary**: Executive TL;DR, detailed summary, bulleted key takeaways, and structured curriculum chapters (**sections**).
- **KeyMoment**: Visual timestamps, labels, descriptions, and importance scores.
- **Quiz**: Multiple-choice questions, options arrays, correct answer indices, and educational explanations.
- **FlashcardSet**: Concept-definition card pairs linked to timestamps.
- **Setting**: User preferences, **google_drive_token**, **storage_target**, theme preferences.

2.3 API Route Controllers (BACKEND/app/routers/)

- **auth.py**: Handles user registration (**/register**), credential verification and JWT token issuance (**/login**), Google Identity Services OAuth token login (**/google**), and user profile introspection (**/me**).
- **videos.py**: Handles multipart video uploads (**/upload**), YouTube video extraction (**/youtube**), catalog listing (**/**), single video intelligence retrieval (**/id**), HTTP 206 partial content streaming with Google Drive proxy (**/id/stream**), and cascading deletion (**/id**).
- **summaries.py**: Handles summary retrieval and interactive AI Concept Mind Map graph synthesis (**/video_id/mindmap**).
- **educator.py**: Endpoints for the 5-Tab Educator Studio: updating transcript segments and speaker tags, saving curriculum chapters, saving custom quiz questions, and saving flashcard decks.
- **learner.py**: Endpoints for student study: retrieving quizzes, submitting answers for automated scoring, and saving user bookmarks.
- **settings.py**: Endpoints for managing user preferences, storage mode toggling, and Google Drive connection status (**/storage/status**, **/storage/mode**).
- **admin.py**: Administrative endpoints for user management, role reassignment, system telemetry, 50+ event audit logs, and cache purging.
- **websocket.py**: WebSocket endpoint (**/ws/videos/video_id**) managing real-time telemetry subscriptions during video processing.

2.4 Media & AI Processing Services (BACKEND/app/services/)

- **drive_storage.py**: Interfaces with Google Drive API v3 to provide resumable multipart uploads, folder auto-provisioning, file metadata retrieval, and direct media downloading for background processing.
- **google_auth.py**: Validates Google OAuth2 tokens and ID tokens issued by Google Identity Services (GIS).
- **downloader.py**: Invokes **yt-dlp** in non-blocking mode to download audio and video streams from YouTube URLs.
- **ffmpeg_extractor.py**: Runs system FFmpeg binaries to demux audio into 16kHz mono WAV format (**pcm_s16le**), perfectly matching Whisper's expected acoustic format.
- **whisper_stt.py**: Loads OpenAI Whisper transformer model with automatic hardware acceleration (CUDA GPU or multithreaded CPU), outputting word-level and sentence-level timestamped transcripts.
- **nlp_summarizer.py**: Computes LexRank eigenvector graph centrality over TF-IDF cosine sentence similarities, generating deterministic extractive summaries in under 1.5 seconds.
- **llm_service.py**: Connects to instruction-tuned LLMs (Groq LPU, Google Gemini, or local models) to generate abstractive chapters, takeaways, quizzes, and flashcards.
- **keyframe_extractor.py**: Samples video frames at 1 fps using OpenCV (**cv2.VideoCapture**), computing pixel delta vectors and HSV histogram distances to detect slide transitions, saving WebP thumbnails.
- **exporter.py**: Generates publication-grade PDF study guides (ReportLab), editable Word documents (**python-docx**), plain text (TXT), and subtitle caption files (SRT, VTT).
- **pipeline/__init__.py**: Orchestrates the multi-stage background pipeline with automatic Google Drive download fallback for cloud media, and broadcasts real-time progress events over WebSockets.

3. Frontend Architecture (FRONTEND/)

Built with **React 19**, **TypeScript 5.5**, and **Vite 8.2** for fast compilation (1.56s production builds) and smooth client-side interactions.

3.1 UI Components (FRONTEND/src/components/)

- **DashboardShell.tsx**: Unified app shell featuring human-crafted navigation, theme switcher (Dark/Light), role badge, and mobile navigation drawer.
- **Navbar.tsx**: Navigation header with user profile menu, storage status indicator, and quick-action buttons.
- **AuthPage.tsx**: Modern authentication modal supporting one-click Google Sign-in and role switcher credentials.
- **UploadStudio.tsx**: Drag-and-drop file upload zone, YouTube URL input, and persistent Google Drive cloud storage toggle.
- **MindMapView.tsx**: Interactive AI Concept Mind Map canvas rendering hierarchical knowledge graphs with pan, zoom, click-to-seek, and SVG export.
- **VideoGrid.tsx & VideoCard.tsx**: Responsive video catalog displaying processing status pills, duration badges, and action dropdowns.
- **VideoIntelligence.tsx**: Main study room layout uniting the HTML5 video player, time-aligned transcript viewer, summary breakdown, mind map tab, and key moments timeline.
- **EducatorEditor.tsx**: 5-Tab authoring studio for educators:
 - *Tab 1*: Transcript editor with speaker diarization.
 - *Tab 2*: Curriculum chapters and time-boundary builder.
 - *Tab 3*: Multiple-choice quiz builder with explanations.
 - *Tab 4*: Active recall flashcard deck builder.
 - *Tab 5*: Live student preview mode.
- **LearnerDashboard.tsx**: Dedicated student interface with synchronized video playback, auto-scrolling transcripts, and interactive study modules.
- **QuizModal.tsx**: Interactive quiz interface allowing learners to submit answers and see instant scoring with pedagogical explanations.
- **FlashcardViewer.tsx**: 3D flip card component for spaced-repetition active recall practice.
- **AdminDashboard.tsx**: Administrative control panel displaying system health metrics, registered users, 50+ event audit logs, and cache purge buttons.
- **ExportModal.tsx**: Dialog enabling users to download PDF, DOCX, TXT, SRT, or VTT files.

3.2 Services & API Client (FRONTEND/src/services/)

- **api.ts**: Configured Axios instance with request/response interceptors that automatically attach JWT bearer tokens and handle 401/403 errors.
- **websocket.ts**: WebSocket client that connects to `/ws/videos/{id}` to receive real-time progress events and trigger UI updates.

4. Documentation & Automated Generation Assets (docs/)

- **docs/COMPLETE_PROJECT_DOCUMENTATION.md**: Master engineering and academic report (16 comprehensive sections).
- **docs/COMPLETE_PROJECT_DOCUMENTATION.docx**: Formatted Microsoft Word document with embedded diagrams and styled tables.
- **docs/COMPLETE_PROJECT_DOCUMENTATION.pdf**: Formatted Adobe PDF document with running headers, footers, and page numbers.
- **docs/ClipMind_AI_Final_Presentation.pptx**: 10-slide high-impact, diagrammatic presentation deck generated via `generate_light_ppt.py`.
- **docs/convert_doc.py**: Automated conversion script parsing Markdown into publication-grade **.docx** and **.pdf** files.
- **docs/generate_light_ppt.py**: Automated script assembling the presentation deck with custom geometric flowcharts and cards.